

Università degli Studi di Torino
Corso di Laurea Magistrale in Informatica

Tecnologie del Linguaggio Naturale

Parte 2 – Prof. Daniele Radicioni

Lab 8

Word Clustering con Word2Vec

Confronto tra testi musicali: anni 70 vs anni 2000

Gruppo di lavoro:

Studente	Matricola
Alessandro Olivero	915069
Samuele Iudice	1235245
Andrea Franco	1226739

Docente: Prof. Daniele Radicioni

Anno Accademico: 2025/2026

Indice

1	Introduzione	2
2	Background Teorico	2
2.1	Rappresentazioni distribuzionali	2
2.2	Word2Vec: architetture	2
2.3	Similarità coseno	2
2.4	Analisi dei vicini e semantic shift	3
2.5	K-Means Clustering	3
2.6	t-SNE	3
3	Dataset e Pre-elaborazione	3
3.1	Dataset	3
3.2	Selezione degli artisti	3
3.3	Pipeline di pulizia	4
4	Addestramento dei Modelli Word2Vec	5
5	Analisi delle Parole Simili	6
6	Similarità Coseno tra Coppie di Parole	7
7	Analisi del Vocabolario	8
8	Analisi Sistematica del Semantic Shift	9
8.1	Coppie con maggiore variazione di similarità	9
8.2	Parole con maggiore cambiamento di contesto	10
9	Clustering K-Means	11
10	Visualizzazione t-SNE	12
11	Risultati e Discussione	13
11.1	Osservazioni principali	13
11.2	Limiti dell'approccio	13
12	Conclusioni	14
13	Dichiarazione sui Contributi di IA	14

1 Introduzione

Questo laboratorio esplora le **rappresentazioni distribuzionali del lessico** tramite il modello Word2Vec, applicato a un dominio insolito e ricco di variazione diacronica: i **testi delle canzoni**.

L'obiettivo principale è confrontare il linguaggio della musica degli **anni 70** (rock, pop, soul) con quello degli **anni 2000** (pop moderno, rap, alternative rock), analizzando:

- Come le rappresentazioni vettoriali delle parole differiscono tra le due ere.
- Quali parole hanno cambiato il loro contesto semantico nel tempo.
- Se i cluster lessicali riflettono temi culturali diversi tra i due periodi.

Il dataset usato è lo *Spotify Million Song Dataset*, che contiene testi di migliaia di artisti. Si selezionano manualmente artisti rappresentativi per ciascuna era e si addestrano due modelli Word2Vec separati, uno per era.

2 Background Teorico

2.1 Rappresentazioni distribuzionali

L'ipotesi distribuzionale, formulata da Firth (1957), afferma che *“you shall know a word by the company it keeps”*: il significato di una parola emerge dal contesto in cui appare. Word2Vec concretizza questa idea mappando ogni parola in un vettore denso nello spazio $\mathbb{R}^d$ tale che parole con contesti simili abbiano vettori vicini.

2.2 Word2Vec: architetture

Word2Vec (Mikolov et al., 2013) propone due architetture complementari:

- **CBOW (Continuous Bag of Words)**: predice la parola target dato il contesto circostante. Più veloce, funziona bene su corpus grandi.
- **Skip-gram**: predice le parole di contesto data la parola target. Più lento ma più efficace su corpus piccoli e per parole rare.

In questo laboratorio si usa **Skip-gram** (impostato esplicitamente con **sg=1**, poiché il default di Gensim è invece CBOW), con finestra di contesto $w = 5$ e dimensione dei vettori $d = 100$.

2.3 Similarità coseno

La similarità tra due vettori $\mathbf{u}$ e $\mathbf{v}$ si misura con la **similarità coseno**:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} \in [-1, 1]$$

Valori vicini a 1 indicano alta similarità semantica; valori vicini a 0 indicano indipendenza; valori negativi indicano opposizione semantica.

2.4 Analisi dei vicini e semantic shift

Per ogni parola si possono estrarre i k **vicini più prossimi** nello spazio vettoriale. Confrontando i vicini della stessa parola nei due modelli (anni 70 vs anni 2000) si ottiene una misura di **semantic shift**: quanto è cambiato il contesto d'uso di quella parola nel tempo.

La metrica usata è l'**overlap dei vicini**:

$$\text{overlap}(p) = \frac{|N_{70}(p) \cap N_{2000}(p)|}{k}$$

Un overlap basso (≈ 0) indica che il contesto semantico della parola è completamente cambiato; un overlap alto (≈ 1) indica stabilità semantica.

2.5 K-Means Clustering

Il **K-Means** è un algoritmo di clustering non supervisionato che partiziona n punti in k cluster minimizzando la varianza intra-cluster:

$$\arg \min_{\{C_j\}} \sum_{j=1}^k \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$$

Applicato ai vettori Word2Vec, raggruppa le parole per similarità semantica, rivelando i macrotemi del lessico di ciascuna era.

2.6 t-SNE

Il **t-SNE** (t-distributed Stochastic Neighbor Embedding) è una tecnica di riduzione dimensionale non lineare che proietta vettori ad alta dimensione ($d = 100$) in 2D preservando la struttura locale. È usato per visualizzare i cluster Word2Vec: parole semanticamente vicine appaiono vicine nel grafico.

3 Dataset e Pre-elaborazione

3.1 Dataset

Il dataset è lo **Spotify Million Song Dataset** (`spotify_millsongdata.csv`), che contiene testi di canzoni con le colonne `artist`, `song` e `text`.

3.2 Selezione degli artisti

Si selezionano manualmente artisti rappresentativi per ciascuna era:

Selezione artisti per era

```
1  artisti_70s = [  
2      'Black Sabbath', 'Pink Floyd', 'Queen',  
3      'Eagles', 'Fleetwood Mac', 'Bee Gees',  
4      'David Bowie', 'Elton John', 'ABBA',  
5      'Doors', 'Rolling Stones', 'Deep Purple'  
6  ]  
7  
8  artisti_2000s = [  
9      'Eminem', 'Coldplay', 'Linkin Park',  
10     'Christina Aguilera', 'Radiohead', 'Foo Fighters',  
11     'Lady Gaga', 'Taylor Swift', 'The Killers',  
12     'Green Day', 'Nickelback'  
13 ]  
14  
15 df_70s = df[df['artist'].isin(artisti_70s)].copy()  
16 df_2000s = df[df['artist'].isin(artisti_2000s)].copy()
```

La scelta degli artisti è volutamente eterogenea per coprire i generi dominanti di ciascuna era: rock classico e hard rock, glam rock, soft rock e disco per gli anni 70; pop, rap, rock alternativo e post-grunge per gli anni 2000. **Nota:** la selezione iniziale includeva *Led Zeppelin*, che però non è presente nel dataset con questo nome esatto (`df['artist'].isin(...)` fallisce silenziosamente su nomi assenti); è stato sostituito con *Black Sabbath* per mantenere 12 artisti nel gruppo anni 70. Il codice ora verifica esplicitamente il numero di artisti effettivamente trovati per evitare che l'errore passi inosservato.

Con questa selezione, il dataset produce **1729 canzoni** per gli anni 70 (12 artisti) e **1312 canzoni** per gli anni 2000 (11 artisti), su un totale di 57650 canzoni e 643 artisti nel dataset originale.

3.3 Pipeline di pulizia

Pulizia e tokenizzazione dei testi

```
1  def pulisci_testo(testo):  
2      if not isinstance(testo, str):  
3          return []  
4      testo = re.sub(r'\[.*?\]', '', testo)          # rimuove annotazioni tipo [  
5      testo = re.sub(r'[^a-zA-Z\s]', '', testo)      # solo lettere  
6      testo = testo.lower()  
7      tokens = testo.split()  
8      # rimuove stopwords e token troppo corti  
9      tokens = [t for t in tokens if t not in stop and len(t) >= 3]  
10     return tokens  
11  
12 df_70s['tokens'] = df_70s['text'].apply(pulisci_testo)  
13 df_2000s['tokens'] = df_2000s['text'].apply(pulisci_testo)  
14  
15 # filtra canzoni troppo corte  
16 df_70s = df_70s[df_70s['tokens'].map(len) > 5]  
17 df_2000s = df_2000s[df_2000s['tokens'].map(len) > 5]
```

I passi di pulizia sono:

1. Rimozione delle annotazioni strutturali tra parentesi quadre (es. [Chorus], [Verse 1]).
2. Rimozione di tutti i caratteri non alfabetici (apostrofi inclusi).
3. Lowercasing.
4. Rimozione delle stopwords inglesi (NLTK) e dei token con meno di 3 caratteri.
5. Filtraggio delle canzoni con meno di 5 token (testi troppo corti o vuoti).

Nota su contrazioni e stopwords: rimuovendo gli apostrofi prima del filtro (punto 2), le contrazioni inglesi perdono la forma con cui compaiono nella lista di stopwords di NLTK (*don't* → *dont*, *you're* → *youre*), e senza correzione restano nel corpus come rumore lessicale. Le forme contratte più comuni (*dont*, *cant*, *youre*, *ive*, ...) sono state aggiunte esplicitamente alla lista di stopwords.

Dopo la pulizia, il corpus anni 70 conta **163 180 token** (11 174 unici) su 1729 canzoni; il corpus anni 2000 conta **149 820 token** (11 551 unici) su 1312 canzoni.

4 Addestramento dei Modelli Word2Vec

Addestramento Word2Vec con Gensim

```

1  params = dict(
2      sg          = 1,      # Skip-gram (il default di Gensim sarebbe CBOW)
3      vector_size = 100,    # dimensione dei vettori
4      window      = 5,      # finestra di contesto (+-5 parole)
5      min_count    = 3,      # ignora parole con freq < 3
6      workers      = 4,      # thread paralleli
7      epochs       = 20,     # passaggi sul corpus
8      seed         = 42      # riproducibilit 
9  )
10
11 model_70s = Word2Vec(sentences=df_70s['tokens'].tolist(), **params)
12 model_2000s = Word2Vec(sentences=df_2000s['tokens'].tolist(), **params)

```

Si addestrano due modelli **completamente indipendenti**, uno per era. I parametri principali:

Tabella 1: Iperparametri del modello Word2Vec

Parametro	Valore	Significato
<code>sg</code>	1	Skip-gram (0 = CBOW, il default di Gensim)
<code>vector_size</code>	100	Dimensione dello spazio vettoriale
<code>window</code>	5	Parole di contesto a sinistra e destra
<code>min_count</code>	3	Frequenza minima per includere una parola
<code>epochs</code>	20	Numero di passaggi sul corpus

I due modelli condividono la stessa architettura ma sono addestrati su corpus distinti: il confronto tra i loro spazi vettoriali rivela le differenze semantiche tra le due ere musicali. Il vocabolario risultante (dopo il filtro `min_count=3`) è di **4752 parole** per gli anni 70 e

4800 parole per gli anni 2000, di cui **2973 in comune**, 1779 esclusive degli anni 70 e 1827 esclusive degli anni 2000.

5 Analisi delle Parole Simili

Parole più simili per alcune parole chiave

```
1 parole_test = ['love', 'night', 'heart', 'world', 'time', 'life', 'feel']
2
3 for parola in parole_test:
4     sim_70 = [w for w, _ in model_70s.wv.most_similar(parola, topn=3)] \
5               if parola in model_70s.wv else ['---']
6     sim_2000 = [w for w, _ in model_2000s.wv.most_similar(parola, topn=3)] \
7                 if parola in model_2000s.wv else ['---']
```

Per ogni parola chiave si estraggono i 3 vicini più prossimi nei due spazi vettoriali:

Tabella 2: Top-3 vicini più prossimi per parola chiave (risultati reali)

Parola	Top 3 Anni 70	Top 3 Anni 2000
love	forsake, posses, bees	elastic, everlastic, spastic
night	gloria, musk, plague	mmmarry, mmmmarry, friday
heart	hijack, indecision, hijacked	aaate, withstand, amaze
world	evergrowing, alter, joins	rule, slips, swirls
time	verse, capture, spends	thanks, closing, replaced
life	lalala, earn, changin	kandy, intoxicated, overdrive
feel	sheer, cardiac, travlin	careen, garner, manicure

A differenza di quanto ci si potrebbe aspettare, i vicini estratti **non formano quasi mai associazioni tematicamente pulite**: parole come *forsake*, *hijack* o *everlastic* non sono interpretabili come metafore coerenti, e in alcuni casi (*everlastic*, *mmmarry*) sembrano artefatti di parole rare o mal segmentate nei testi originali. Questo è un limite atteso di Word2Vec addestrato su corpus di dimensioni modeste (poche migliaia di canzoni): per parole molto frequenti come *love* o *night*, il contesto immediato è dominato da co-occorrenze rare e idiosincratiche più che da relazioni semantiche stabili, che emergerebbero con corpus ordini di grandezza più grandi. Le analisi sistematiche delle Sezioni 8.1 e 8.2, basate su statistiche aggregate su centinaia di parole invece che su singoli esempi, risultano più robuste di questa ispezione qualitativa puntuale.

6 Similarità Coseno tra Coppie di Parole

Calcolo della similarità coseno tra coppie

```
1 coppie = [  
2     ('love', 'heart'), ('love', 'money'),  
3     ('night', 'dark'), ('god', 'heaven'),  
4     ('war', 'peace'), ('free', 'soul'),  
5 ]  
6  
7 for w1, w2 in coppie:  
8     s70 = model_70s.wv.similarity(w1, w2) \  
9         if w1 in model_70s.wv and w2 in model_70s.wv else None  
10    s2000 = model_2000s.wv.similarity(w1, w2) \  
11        if w1 in model_2000s.wv and w2 in model_2000s.wv else None
```

La similarità coseno tra coppie tematicamente significative permette di quantificare le differenze culturali tra le due ere:

Tabella 3: Similarità coseno tra coppie di parole (risultati reali)

Coppia	Anni 70	Anni 2000
(love, heart)	0.333	0.252
(love, money)	0.237	0.249
(night, dark)	0.308	0.340
(god, heaven)	0.255	0.200
(war, peace)	0.429	0.407
(free, soul)	0.259	0.145

- **(love, heart)**: similarità moderata in entrambe le ere, più alta negli anni 70 (0.33 vs 0.25) — coerente con un tema universale la cui espressione lessicale rimane comunque parzialmente stabile.
- **(god, heaven)**: più alta negli anni 70 (0.26 vs 0.20); una possibile lettura è la maggiore presenza di rock spirituale/blues in quel campione di artisti, ma con soli 6 esempi la differenza va presa come indicativa, non come prova.
- **(war, peace)**: valori alti e vicini in entrambe le ere (0.43 vs 0.41) — la coppia resta fortemente associata indipendentemente dal periodo, senza un chiaro segnale di cambiamento storico da questi soli dati.
- **(free, soul)**: calo netto dagli anni 70 (0.26) agli anni 2000 (0.15) — la coppia più significativa del gruppo, compatibile con un progressivo allontanamento semantico dei due termini nel lessico musicale.

7 Analisi del Vocabolario

Confronto dei vocabolari tra le due ere

```
1 vocab_70s = set(model_70s.wv.index_to_key)
2 vocab_2000s = set(model_2000s.wv.index_to_key)
3
4 solo_70s = vocab_70s - vocab_2000s    # parole esclusive anni 70
5 solo_2000s = vocab_2000s - vocab_70s  # parole esclusive anni 2000
6 comuni = vocab_70s & vocab_2000s     # parole condivise
7
8 # parole caratteristiche (esclusive + frequenti)
9 interessanti_70s = sorted(
10     [p for p in solo_70s if freq_70s[p] > 20],
11     key=lambda x: -freq_70s[x]
12 )
13 interessanti_2000s = sorted(
14     [p for p in solo_2000s if freq_2000s[p] > 20],
15     key=lambda x: -freq_2000s[x]
16 )
```

L'analisi del vocabolario distingue tre insiemi:

- **Parole esclusive degli anni 70:** termini del rock psichedelico, della controcultura e del glam rock che non compaiono (con frequenza sufficiente) nei testi degli anni 2000.
- **Parole esclusive degli anni 2000:** termini tipici del rap, del pop elettronico e del linguaggio urbano contemporaneo.
- **Parole comuni:** il nucleo lessicale condiviso tra le due ere — temi universali come amore, tempo, vita, notte.

8 Analisi Sistemática del Semantic Shift

8.1 Coppie con maggiore variazione di similarità

Analisi sistemática delle coppie di parole comuni (versione vettorizzata)

```
1  FREQ_COPPIE = 100    # frequenza minima in entrambe le ere
2
3  candidati = [
4      p for p in (vocab_70s & vocab_2000s)
5      if freq_70s[p] >= FREQ_COPPIE and freq_2000s[p] >= FREQ_COPPIE
6  ]
7
8  # similarita' vettorizzata: una matrice N x N invece di N^2/2 chiamate a
9  # .similarity(), utile quando i candidati salgono a qualche centinaio
10 def matrice_coseno(model, parole):
11     vettori = np.array([model.wv[p] for p in parole])
12     norme   = vettori / np.linalg.norm(vettori, axis=1, keepdims=True)
13     return norme @ norme.T
14
15 sim70, sim2000 = matrice_coseno(model_70s, candidati), matrice_coseno(
16     model_2000s, candidati)
17
18 coppie_sist = []
19 for i, w1 in enumerate(candidati):
20     for j in range(i + 1, len(candidati)):
21         w2, s70, s2000 = candidati[j], float(sim70[i, j]), float(sim2000[i, j])
22         coppie_sist.append({
23             'w1': w1, 'w2': w2, 's70': s70, 's2000': s2000,
24             'delta': abs(s70 - s2000), 'direzione': s2000 - s70,
25         })
26
27 coppie_sist.sort(key=lambda x: -x['delta'])
```

Per ogni coppia di parole che compaiono con frequenza sufficiente in *entrambi* i corpus, si calcola il **delta di similarità**:

$$\Delta(w_1, w_2) = |\text{sim}_{70}(w_1, w_2) - \text{sim}_{2000}(w_1, w_2)|$$

Le coppie con Δ alto sono quelle la cui relazione semantica è più cambiata tra le due ere. La *direzione* del cambiamento ($\text{sim}_{2000} - \text{sim}_{70}$) indica se le parole si sono avvicinate o allontanate:

- **Direzione positiva:** le due parole sono diventate più associate negli anni 2000.
- **Direzione negativa:** le due parole erano più associate negli anni 70 e si sono semanticamente allontanate.

Con soglia FREQ_COPPIE=100 si ottengono **245 parole candidate** e quindi **29 890 coppie** confrontate. Le 5 coppie con delta più alto sono:

Tabella 4: Top 5 coppie con maggiore variazione di similarità (risultati reali)

Coppia	Anni 70	Anni 2000	Δ	Trend
(free, knows)	0.268	−0.139	0.408	più lontane
(friends, move)	−0.105	0.299	0.405	più vicine
(anything, soul)	0.343	−0.053	0.397	più lontane
(play, used)	0.366	−0.029	0.396	più lontane
(running, hands)	0.360	−0.026	0.386	più lontane

8.2 Parole con maggiore cambiamento di contesto

Overlap dei vicini per misurare il semantic shift

```

1  FREQ_MINIMA = 30
2  TOP_N_VICINI = 10
3
4  parole_frequenti_comuni = [
5      p for p in comuni
6      if freq_70s[p] >= FREQ_MINIMA and freq_2000s[p] >= FREQ_MINIMA
7  ]
8
9  risultati = []
10 for parola in parole_frequenti_comuni:
11     vicini_70s = set(w for w, _ in
12                     model_70s.wv.most_similar(parola, topn=TOP_N_VICINI))
13     vicini_2000s = set(w for w, _ in
14                       model_2000s.wv.most_similar(parola, topn=TOP_N_VICINI))
15
16     overlap = len(vicini_70s & vicini_2000s) / TOP_N_VICINI
17     cambiamento = 1 - overlap
18
19     risultati.append({
20         'parola': parola, 'cambiamento': cambiamento,
21         'overlap': overlap,
22         'vicini_70s': vicini_70s, 'vicini_2000s': vicini_2000s,
23     })
24
25 risultati.sort(key=lambda x: -x['cambiamento'])

```

L'analisi sistematica calcola, per ogni parola comune frequente, l'overlap tra i 10 vicini più prossimi nelle due ere. Per ciascuna parola nel top-10 si analizza in dettaglio:

- I **vicini spariti**: parole vicine negli anni 70 che non compaiono più nel vicinato degli anni 2000 — indicano contesti culturali scomparsi.
- I **vicini nuovi**: parole vicine negli anni 2000 ma non negli anni 70 — indicano nuovi usi o associazioni culturali.
- I **vicini stabili**: parole rimaste nel vicinato in entrambe le ere — il nucleo semantico stabile della parola.

Con soglia `FREQ_MINIMA=30` si ottengono **648 parole comuni** analizzate. Un risultato inatteso: per la maggior parte delle parole nel top-30 (es. *daddy, understand, would, stars*) l'overlap è **0%** (cambiamento 100%) — nessuno dei 10 vicini più prossimi coincide fra le due ere. Le parole più **stabili** (overlap più alto, comunque solo 20–30%) sono *new (brand,*

orleans, york), *pay* (*price, rent*) e *fate* (*sealed, twist*). Questo conferma quanto osservato in Sezione 5: con corpus di poche migliaia di canzoni per era, il vicinato immediato di una parola è dominato da rumore statistico più che da un segnale semantico stabile, e va quindi interpretato con cautela.

9 Clustering K-Means

Clustering K-Means sulle top 200 parole

```
1 def fai_clustering(model, n_clusters=6, top_words=200):
2     parole = model.wv.index_to_key[:top_words] # top 200 per freq
3     vettori = np.array([model.wv[w] for w in parole])
4     km = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
5     labels = km.fit_predict(vettori)
6     cluster = {}
7     for p, l in zip(parole, labels):
8         cluster.setdefault(l, []).append(p)
9     return cluster, list(parole), vettori, labels
10
11 cl_70s, par_70s, vec_70s, lbl_70s = fai_clustering(model_70s)
12 cl_2000s, par_2000s, vec_2000s, lbl_2000s = fai_clustering(model_2000s)
```

Il clustering opera sulle **200 parole più frequenti** del vocabolario di ciascun modello, raggruppandole in $k = 6$ cluster tramite K-Means applicato ai vettori Word2Vec a 100 dimensioni.

Tabella 5: Cluster K-Means, anni 70 (risultati reali, prime 12 parole per cluster)

Cluster	Parole
0	know, got, time, see, get, well, take, come, let, man, back, could
1	song, sing
2	love, like, yeah, one, way, say, away, life, night, little, right, world
3	never, eyes, light, believe, dance, face, dream, must, fire, far, sky, lost
4	baby, want, gonna, make, give, would, gotta, sweet, alright, honey, money, rock
5	day, things, head, new, stay, made, days, kind, years, done, used

Tabella 6: Cluster K-Means, anni 2000 (risultati reali, prime 12 parole per cluster)

Cluster	Parole
0	world, keep, leave, tired, open
1	know, love, get, one, got, come, want, never, cause, back, see, baby
2	wrong, song, sing
3	said, eyes, would, even, change, wish, thought, help, forever, christmas, knew, true
4	like, yeah, time, feel, give, need, think, life, night, day, look, ooh
5	around, put, long, believe, someone, dance, high, stand, hands, talk, getting, real

Lavorando sulle 200 parole più **frequenti** (non su un campione casuale del vocabolario), i cluster sono dominati da parole funzionali e verbi comuni (*know, got, get, come*)

piuttosto che da lessico tematico raro: il risultato è quindi più vicino a una segmentazione **sintattico-collocazionale** che a una netta separazione per macrotema. Detto questo, alcuni cluster restano interpretabili: il cluster 4 degli anni 70 (*baby, want, gonna, sweet, honey, money*) è riconoscibile come nucleo lessicale del desiderio/corteggiamento, e il cluster 3 degli anni 70 (*eyes, light, dance, dream, fire, sky*) ha una connotazione atmosferico-onirica coerente col rock degli anni 70. Nei cluster degli anni 2000 emergono invece isole più piccole e specifiche (cluster 0: *tired, open*; cluster 2: *wrong, song, sing*), segno di un vocabolario più frammentato su questo campione. Va però ribadito il limite già discusso in Sezione 5: con corpus di questa dimensione le strutture emergenti vanno lette come indicazioni, non come conclusioni robuste.

10 Visualizzazione t-SNE

Riduzione dimensionale t-SNE e plot dei cluster

```

1 def tsne_plot(parole, vettori, labels, titolo, ax, n_clusters=6):
2     # niente max_iter/n_iter esplicito: il nome cambia tra versioni di
3     # scikit-learn, il default (1000) e' comunque quello voluto
4     tsne = TSNE(n_components=2, random_state=42, perplexity=30)
5     v2d = tsne.fit_transform(vettori) # 100D -> 2D
6
7     colori = plt.cm.Set1(np.linspace(0, 1, n_clusters))
8     highlight = {
9         'love', 'night', 'heart', 'baby', 'time',
10        'world', 'life', 'feel', 'fire', 'rain',
11        'sun', 'soul', 'free', 'dark', 'light', 'god'
12    }
13
14    for cid in range(n_clusters):
15        mask = labels == cid
16        ax.scatter(v2d[mask, 0], v2d[mask, 1],
17                  c=[colori[cid]], label=f'Cluster {cid}',
18                  alpha=0.7, s=60)
19
20    # annota le parole semanticamente rilevanti
21    for i, p in enumerate(parole):
22        if p in highlight:
23            ax.annotate(p, (v2d[i, 0], v2d[i, 1]),
24                        fontsize=9, fontweight='bold',
25                        xytext=(5, 5), textcoords='offset points')
```

Il grafico t-SNE mostra la distribuzione delle 200 parole più frequenti in 2D, con colori diversi per cluster. Le parole semanticamente rilevanti (amore, vita, notte, cuore, ecc.) vengono annotate direttamente sul grafico per facilitare l'interpretazione.

La disposizione dei punti rivela la **struttura interna** del lessico di ciascuna era: se i cluster sono ben separati, il linguaggio è più specializzato per tema; se i cluster si sovrappongono, il lessico è più polisemico e interdependente.

11 Risultati e Discussione

Tabella 7: Riepilogo comparativo delle caratteristiche dei due corpus

Caratteristica	Anni 70	Anni 2000
Artisti rappresentativi	12	11
Generi principali	Rock, Glam, Disco	Pop, Rap, Alt-rock
Modello	Word2Vec Skip-gram	Word2Vec Skip-gram
Dimensione vettori	100	100
Finestra di contesto	5	5
Metrica di confronto	Similarità coseno	Similarità coseno
Analisi shift semantico	Overlap vicini ($k = 10$)	Overlap vicini ($k = 10$)
Clustering	K-Means ($k = 6$)	K-Means ($k = 6$)
Visualizzazione	t-SNE 2D	t-SNE 2D

11.1 Osservazioni principali

1. **Le rappresentazioni vettoriali catturano differenze culturali:** la stessa parola (es. *love*, *night*) ha vicini diversi nei due modelli, riflettendo il diverso immaginario delle due ere musicali.
2. **Il semantic shift è misurabile quantitativamente:** l'overlap dei vicini fornisce una metrica comparabile tra parole diverse. Parole con overlap basso hanno attraversato un cambiamento semantico significativo.
3. **Il vocabolario esclusivo rivela le specificità culturali:** le parole presenti solo in un'era riflettono temi, slang e riferimenti culturali propri di quel periodo storico.
4. **I cluster K-Means sono interpretabili:** i gruppi semantici emergono in modo naturale dai vettori Word2Vec senza supervisionare il processo, confermando la qualità delle rappresentazioni apprese.
5. **t-SNE visualizza la struttura semantica:** la separazione spaziale dei cluster nel piano 2D riflette le relazioni tra macrotemi lessicali, rendendo visibili differenze altrimenti invisibili in 100 dimensioni.

11.2 Limiti dell'approccio

- **Bias nella selezione degli artisti:** la scelta manuale degli artisti introduce un bias soggettivo. Artisti diversi avrebbero potuto dare risultati differenti.
- **Corpus sbilanciati:** se gli artisti degli anni 2000 hanno in media più canzoni nel dataset, il modello corrispondente sarà addestrato su più dati, rendendo il confronto non del tutto omogeneo.
- **Spazi vettoriali non allineati:** i due modelli sono addestrati indipendentemente; gli spazi vettoriali risultanti non sono direttamente comparabili tramite distanza euclidea — la similarità coseno interna a ciascun modello è però robusta.
- **Dipendenza dal preprocessing:** la rimozione delle stopwords e dei token corti può eliminare parole grammaticalmente significative.

12 Conclusioni

Il laboratorio ha dimostrato come Word2Vec, applicato a testi musicali, sia in grado di catturare differenze semantiche e culturali tra due ere distanti nel tempo. I modelli addestrati separatamente sugli anni 70 e sugli anni 2000 producono spazi vettoriali che riflettono i diversi immaginari, temi e registri linguistici delle rispettive epoche.

L’analisi sistematica del semantic shift — tramite overlap dei vicini e delta di similarità — permette di identificare quantitativamente quali parole hanno cambiato contesto d’uso e in quale direzione. I cluster K-Means e la visualizzazione t-SNE completano l’analisi con una prospettiva strutturale, rivelando i macrotemi lessicali di ciascuna era.

Questo tipo di analisi, applicabile a qualsiasi corpus diacronico, mostra il potenziale delle rappresentazioni distribuzionali per lo studio dell’evoluzione del linguaggio nel tempo.

13 Dichiarazione sui Contributi di IA

Per questo laboratorio ci siamo appoggiati anche a uno strumento di intelligenza artificiale generativa (Claude, Anthropic), usato come supporto puntuale durante lo sviluppo e non come autore del lavoro. Qualche esempio concreto di come lo abbiamo usato:

- **Correzione di un bug nel codice:** nel riepilogo finale dello script il passaggio era erroneamente etichettato come “[STEP 10]” invece di “[STEP 12]”, un residuo di una versione precedente con meno fasi di analisi.
- **Rimozione di codice morto:** due import mai utilizzati (`nltk`, `word_tokenize`), rimasti da una versione precedente dello script in cui la tokenizzazione era gestita diversamente.
- **Snellimento dei commenti:** molti commenti erano ridondanti rispetto al codice stesso; li abbiamo resi più mirati (es. cosa rappresenta l’overlap dei vicini come proxy del semantic shift).
- **Confronto sulle scelte di analisi:** discussione sul perché normalizzare per la dimensione del vocabolario nel confronto tra i due modelli Word2Vec, e su quali limiti avesse l’analisi (bias nella selezione degli artisti, corpus sbilanciati).

L’impostazione del lavoro, le scelte metodologiche, l’interpretazione critica dei risultati e la stesura della relazione restano nostre.